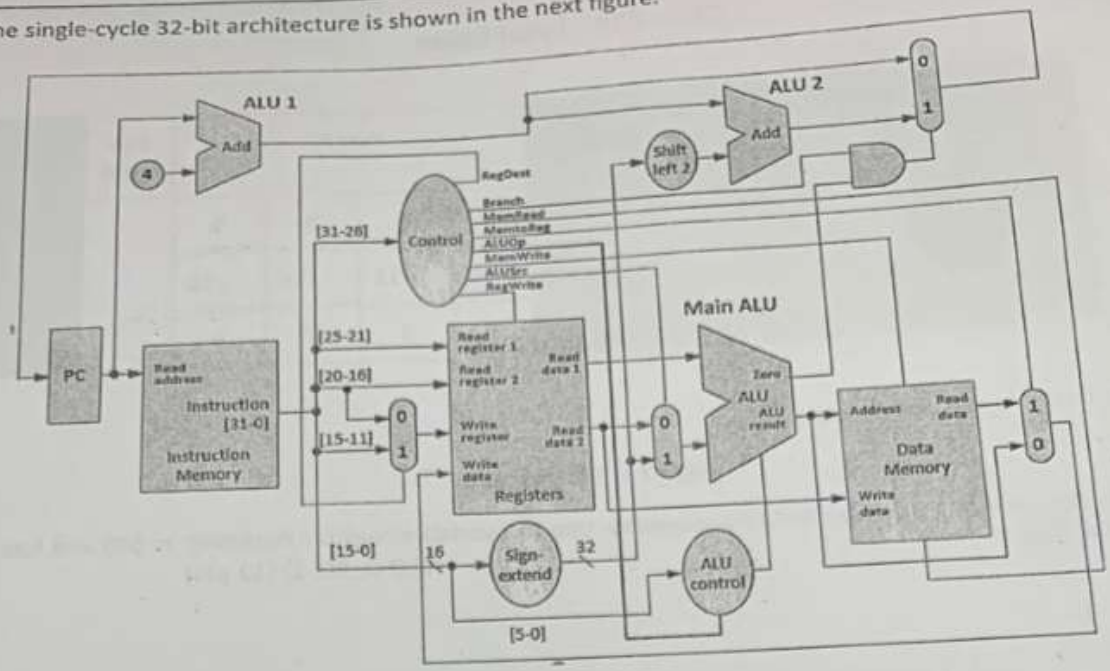
3. The single-cycle 32-bit architecture is shown in the next figure. (CLO-8, SO-1)




Assume that the following instruction is executed on this architecture.

```
lw $s3, 16($s1)
```

a. How many bits are used to store the constant 16 in the instruction above? Is it required to convert this to use additional bits? If so, specify the functional units in the figure which are needed to achieve this. What is the role of "Shift left 2" in the execution of this instruction? (4 points)

b. What is the use of ALU1 and ALU2 for this instruction? (3 points)

c. Is Zero of the main ALU used in this instruction? If not, which output of the main ALU is used in this instruction?

d. What are the values of these control elements for this instruction? (2 points)

| MemtoReg | MemRead | RegDest | ALUSrc |
|---|---|---|---|
| | | | |

4. The following assembly program is executed on a 5-stages (IF, ID, EXE, MEM, WB) pipelined architecture with two separate instruction and data memories: (CLO-10, SO-1) (12 pts)

```
1: add $s1, $s2, $s3
2: lw $t1, 8($s2)
3: add $t3, $s1, $t2
4: sw $t1, 12($s2)
```

a. Draw a pipelined diagram (using blocks) with all necessary stalls and forwarding paths.

b. What is the total number of clock cycles needed to execute this program?

5. a. In the following MIPS instruction sequence, **which type of hazard** is produced in a pipelined system? and **at which instruction**? (CLO-11, SO-1) (4 points)

```
lw $s2, 10($s1)
sw $s3, 20($s1)
bne $s0, $zero, Exit
add $s4, $s3, $s6
…
Exit: …
```

b. In general, **what causes this type of hazard** in a pipelined data path, and **how can be eliminated**? (4 points)

c. Propose a solution to avoid this type of hazard and explain how it can be done. (2 points)

## Part II – 50 points

6. What are the contents of $t0 and $t2 after the four indicated instructions? (CLO-3, SO-1) (10 pts)

```
addi $t0, $zero, 8        # instruction1, $t0 =       , $t2 =
addi $t2, $zero, -4       # instruction2, $t0 =       , $t2 =
sll  $t2, $t0,   2        # instruction3, $t0 =       , $t2 =
bne  $t2, $t0,   ELSE
j    DONE

ELSE: addi $t2, $t2, -3

DONE:                     # instruction4, $t0 =       , $t2 =
```

7. Write a method(procedure) that implements the ***min*** operation using MIPS code. The ***min*** method takes two numbers as input parameters and returns the smaller of the two numbers. If the numbers are equal then either of the two numbers could be returned. Do not use pseudo-instructions. (CLO-4, SO-1) (12 pts)

8. Answer the following questions. (CLO-7, SO-1) (18 pts)

a) Explain with a suitable example how pseudo instructions help a developer code faster. (6 points)

b) Illustrate with an example how the *mult* command in MIPS handles signed multiplication. (6 points)

c) Can a *for-loop* used in high-level languages be implemented in MIPS? Explain with an appropriate example. (6 points)

# CPCS 214 - Final Exam

| | Part I (SO-1) | | | | | Sub total | Part II | | | | Sub total | Total |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Q | 1 | 2 | 3 | 4 | 5 | | 6 | 7 | 8 | 9 | | |
| Pts | /12 | /6 | /10 | /12 | /10 | | /10 | /12 | /18 | /10 | | |
| CLO | 2 | 5 | 8 | 10 | 11 | | 3 | 4 | 7 | 9 | | |

Answer all questions. Total points : 100

## Part I – 50 points

1. Compile the following program to MIPS assembly. Use the variable-register mapping: n: $t0 and base register of A: $s0. (CLO-2, SO-1) (12 pts)

```
int x = 1;
while (x<7)
{
      A[x+1] = A[x] + 4;
      x++;
}
```

2. Convert the number: (-87.75), to the standard IEEE single-precision format. (CLO-5, SO-1) (6 points)